UNIT IV

What is STL?

Standard Template Library (STL)

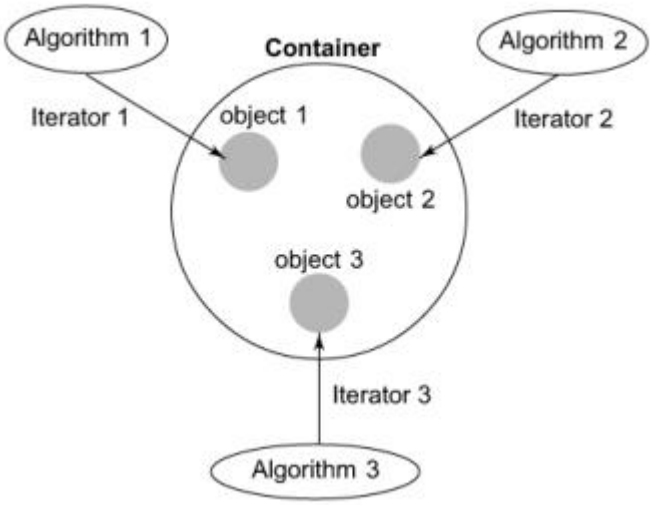
- **This collection of generic classes(Data Structures) and functions (algorithms) is called the Standard Template Library (STL).**
- **The STL has now become a part of the ANSI standard C++ class library.**
- **STL is large and complex, but its most important features enable users to begin using it effectively.**
- **Using STL can save considerable time and effort, and lead to high quality programs.**
- **These benefits are possible because we are basically reusing well-written and well-tested components defined in the STL.**
- **STL components are defined in the namespace std.**
- **We must therefore use the directive:**

using namespace std;

to inform the compiler that we intend to use the Standard C++ Library.

Components of STL

- **The STL contains several components, but at its core are three key components:**
  1. **Containers**
  2. **Algorithms**
  3. **Iterators**
- **These three components work in conjunction with one another to provide support to a variety of programming solutions.**
- **Algorithms employ iterators to perform operations stored in containers.**



Container	Algorithm	Iterator
Stores and organises data (vector, list, stack, map…)	Performs operations on data (sort, search, reverse…)	Points to elements inside a container (like a cursor)

- A **container** is an object that actually stores data. It is a way data is organized in memory.
- The STL containers are implemented by **template classes** and therefore can be easily customized to hold different types of data.
- An **algorithm** is a procedure that is used to process the data contained in the containers.
- The STL includes many different kinds of algorithms to provide support to tasks such as **initializing, searching, copying, sorting, and merging**.
- Algorithms are implemented by **template functions**.
- An **iterator** is an object (like a pointer) that points to an element in a container.
- We can use iterators to move through the contents of containers.
- Iterators are handled just like pointers. We can **increment or decrement them**.
- Iterators connect algorithms with containers and play a key role in the manipulation of data stored in the containers.

Container:

- As stated earlier, containers are objects that hold data (of same type).
- The STL defines **ten containers** which are grouped into **three categories**:
    1. **Sequence containers**
    2. **Associative containers**
    3. **Derived containers (container adaptors)**
- Each container class defines a set of functions that can be used to manipulate its contents.
- Example: A **vector container** defines functions for inserting elements, erasing the contents, and swapping the contents of two vectors.

📌 Sequence Containers

- Sequence containers store elements in a **linear sequence**, like a line.
- Each element is related to other elements by its position along the line.
- They expand themselves to allow insertion of elements and support a number of operations.
- STL provides **three types of sequence containers**:
    - **vector**

    - **list**
    - **deque**
- Elements in all these containers can be accessed using an iterator.
- The difference between them is related only to **performance** (random access vs insertion/deletion speed).

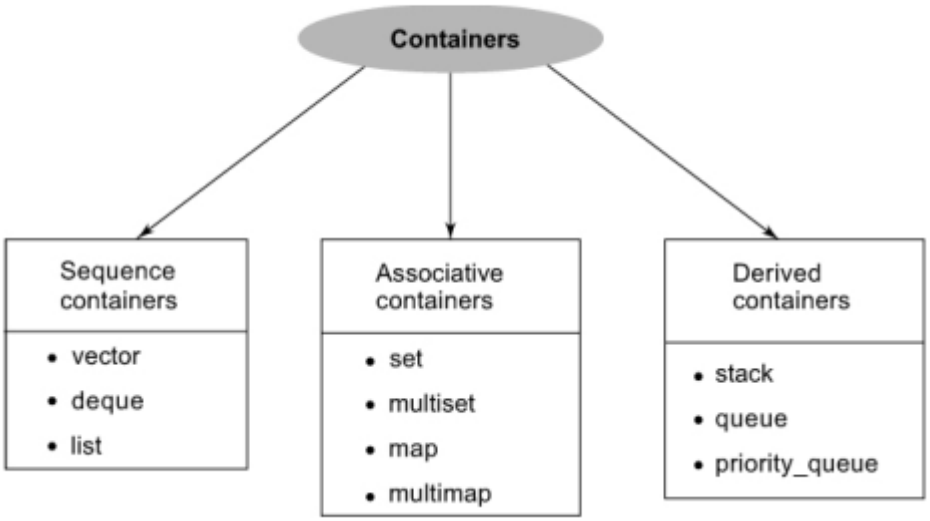
📌 **Associative Containers**

- Associative containers support **direct access to elements using keys**.
- They are **not sequential**.
- STL provides **four types of associative containers**:
    - **set**
    - **multiset**
    - **map**
    - **multimap**
- All these containers store data in a **tree structure** which facilitates fast searching, deletion, and insertion.
- They are slow for random access and inefficient for sorting.
- **set vs multiset**:
    - Set does not allow duplicate items.
    - Multiset allows duplicate items.
- **map vs multimap**:
    - Map allows only one key for a given value.
    - Multimap permits multiple keys for a given value.

📌 **Derived Containers (Container Adaptors)**

- STL provides **three derived containers**:
    - **stack**
    - **queue**
    - **priority_queue**
- These are also known as **container adaptors**.
- They can be created from different sequence containers.
- Derived containers do **not support iterators**, so they cannot be used for direct data manipulation.
- They support two member functions:

- **push()** → for inserting
- **pop()** → for deleting



Vector

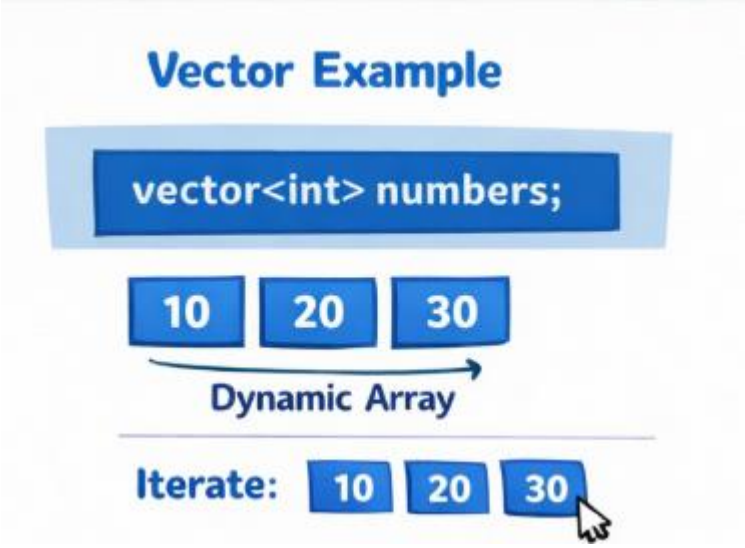
A dynamic array that can grow or shrink automatically

Real-life Analogy

> Imagine a waiting list at a restaurant. You can add a name at the end, remove a name, or check how many people are waiting and the list adjusts its size automatically!

Key Points

- Header file: #include <vector>
- Type: Sequence container (like a dynamic array)
- Automatically resizes when elements are added or removed
- Elements stored in continuous memory — fast to access by index
- Supports random access: v[0], v[1]…



Common Methods

Method	What it does	Example
`push_back(x)`	Adds element x at the end	`v.push_back(5)`
`pop_back()`	Removes the last element	`v.pop_back()`
`size()`	Returns number of elements	`v.size()`
`v[i]`	Access element at index i	`v[0], v[1]`
`empty()`	Returns true if vector is empty	`v.empty()`

Program 1: Create a C++ program using Vector that stores the numbers 10,20,30,40 and print all the elements.

```
#include<iostream>
#include<vector>
using namespace std;

int main() {
    // Create a vector of integers
    vector<int> v;

    // Add elements at the end
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);

    // Print all elements using index
    cout << "Vector elements: ";
```

```
    for(int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
    cout << endl;

    // Remove last element
    v.pop_back();
    cout << "After pop_back, size = " << v.size() << endl;

    return 0;
}
```

▸ Output

```
Vector elements: 10 20 30 40
After pop_back, size = 3
```

Program 2: **Create a C++ program using Vector that stores the numbers 1,2,3,4,5 and print all the elements. Using Foe each loop.**

```
#include <iostream>
#include <vector>


using namespace std;

int main() {

    // create vector of int type
    vector<int> numbers {1, 2, 3, 4, 5};


    // print vector elements using ranged loop
    for (int number : numbers) {
        cout << number << "  ";
    }

    return 0;
}
```

Program 3: Create a C++ program using Vector that stores the numbers 1,2,3,4,5 sort the elements in vector using Sort algorithm and print it in ascending order

```
#include <iostream>
```

```
#include <algorithm>
#include <vector>

using namespace std;

int main() {

    // initialize vector of int type
    vector<int> numbers = {3, 2, 5, 1, 4};

    // sort vector elements in ascending order
    sort(numbers.begin(), numbers.end());



    // print the sorted  vector
    for (int number : numbers) {
        cout << number << "  ";
    }

    return 0;
}
```

Program 4: **Create a C++ program using Vector that stores the numbers 1,2,3,4,5 sort the elements in vector using Sort algorithm and print it in descending order(Highest no first to Lowest number last)**

to sort the elements in descending order:

```
#include <iostream>
#include <vector>
#include <algorithm>
Using namaespace std;

int main() {
    // Create a vector of integers
    vector<int> numbers = {5, 2, 8, 1, 9, 3};

    // Print original vector
    std::cout << "Original vector: ";
    for (int num : numbers) {
        std::cout << num << " ";
    }
```

```
    // Sort the vector in ascending order
sort(numbers.begin(), numbers.end());

    // Print sorted vector
cout << "Sorted vector (ascending): ";
    for (int num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    // Sort the vector in descending order
sort(numbers.begin(), numbers.end(),greater<int>());

    // Print vector sorted in descending order
    std::cout << "Sorted vector (descending): ";
    for (int num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Program 4: Create a C++ program using Vector that stores the numbers 1,2,3,4,5 and orint it using iterator concept.

```
#include<iostream>
#include<vector>
using namespace std;

int main() {
  vector < int > v = {  1, 2, 3, 4, 5 };
  vector < int > ::iterator itr;

  for (itr = v.begin(); itr != v.end(); ++itr) {
    cout << ( * itr) << " ";
  }
  return 0;
}
```

List

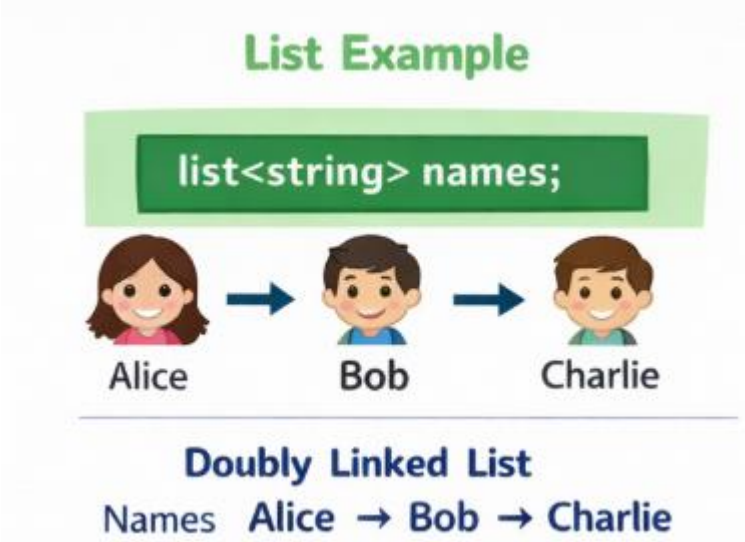
A doubly linked list — efficient insertion/deletion anywhere

Real-life Analogy

> Think of a chain of train bogies. You can attach a bogie at the front, at the back, or in the middle easily — without moving everything else. That's how a list works!

Key Points

- Header file: #include <list>
- Type: Sequence container (doubly linked list)
- Each element is connected to the next and previous element
- Unlike vector, you cannot access elements by index (no l[0])
- Best for frequent insertions and deletions in the middle



Common Methods

Method	What it does
`push_back(x)`	Adds element x at the end
`push_front(x)`	Adds element x at the front
`pop_back()`	Removes the last element
`pop_front()`	Removes the first element
`size()`	Returns number of elements

Program : 1 Create a C++ program using List that stores the numbers 10,20,30, and print it using for each loop, after printing remove the numbers from List.

```
#include<iostream>
#include<list>
using namespace std;

int main() {
    // Create a list of integers
    list<int> l;

    // Add elements
    l.push_back(20);   // add at end   → [20]
    l.push_back(30);   // add at end   → [20, 30]
    l.push_front(10);  // add at front → [10, 20, 30]
    l.push_front(5);   // add at front → [5, 10, 20, 30]

    // Print all elements using range-based for loop
    cout << "List elements: ";
    for(auto x : l) {
        cout << x << " ";
    }
    cout << endl;

    // Remove from front and back
    l.pop_front();
    l.pop_back();
    cout << "After removals: ";
    for(auto x : l)
        cout << x << " ";
    cout << endl;

    return 0;
}
```

▸ Output

```
List elements: 5 10 20 30
After removals: 10 20
```

Step-by-step Trace

Statement	List contents (front → back)
`push_back(20)`	`[20]`
`push_back(30)`	`[20, 30]`
`push_front(10)`	`[10, 20, 30]`
`push_front(5)`	`[5, 10, 20, 30]`

Statement	List contents (front → back)
`pop_front()`	`[10, 20, 30]`
`pop_back()`	`[10, 20]`

 Tip: Use auto x : l (range-based for loop) to print list elements since list does not support index access like l[0].

Program 2: Write a C++ program to store three names , Alice, Bob, Charlie into list and pront all the values using for each loop

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<string> names;   // list container

    names.push_back("Alice");
    names.push_back("Bob");
    names.push_back("Charlie");


    cout << "Names: ";
    for(string n : names) {
        cout << n << " ";
    }
    return 0;
}
```

👉 Output:

```
Code
Names: Alice Bob Charlie
```

Stack

Last In, First Out (LIFO) — like a stack of plates

Real-life Analogy

 Think of a stack of plates in a cafeteria. You always pick the plate from the top, and you always place a new plate on top. The last plate placed is the first one taken out. That is LIFO!

Key Points

- Header file: #include <stack>

- Type: Container Adaptor (built on top of deque/vector)
- LIFO = Last In, First Out
- You can only add/remove from the top
- Cannot access middle elements directly
- No iterator supported

Common Methods

Method	What it does
`push(x)`	Adds element x on top of stack
`pop()`	Removes the top element
`top()`	`Returns the top element (does NOT remove it)`
`empty()`	Returns true if stack is empty
`size()`	Returns number of elements

Program : Write a program in C++ that stores 10,20,20 in the Stack

```
#include<iostream>
#include<stack>
using namespace std;

int main() {
    // Create a stack of integers
    stack<int> s;

    // Push elements onto the stack
    s.push(10);   // Stack: [10]  ← bottom
    s.push(20);   // Stack: [10, 20]
    s.push(30);   // Stack: [10, 20, 30]  ← top

    cout << "Top element: " << s.top() << endl;
    cout << "Stack size: "  << s.size() << endl;

    // Pop elements one by one (LIFO order)
    cout << "Popping elements: ";
    while(!s.empty()) {
        cout << s.top() << " ";   // print top
        s.pop();                  // remove top
    }
    cout << endl;

    return 0;
```

}

▶ Output

Top element: 30

Stack size: 3

Popping elements: 30 20 10

Step-by-step Trace

Statement	Stack (bottom → top)	top()
`push(10)`	`[10]`	10
`push(20)`	`[10, 20]`	20
`push(30)`	`[10, 20, 30]`	30
`pop()`	`[10, 20]`	20
`pop()`	`[10]`	10
`pop()`	`[ empty ]`	—

 Tip: Notice elements come out in reverse order — 30, 20, 10. This is LIFO in action!

Map

Stores data as key-value pairs — like a dictionary

Real-life Analogy

 A dictionary! Every word (key) has a meaning (value). You look up a word and get its meaning. In a map, every key is unique and maps to exactly one value.

Key Points

- Header file: #include <map>
- Type: Associative container
- Stores data as key → value pairs
- Keys are always unique and stored in sorted order
- Access value using key: m["name"]
- Search complexity: O(log n)

Common Methods

Method / Syntax	What it does
`m["key"] = value`	Insert or update a key-value pair
`m.find("key")`	Returns iterator to key (or end() if not found)
`m.erase("key")`	Removes the key-value pair
`m.size()`	Returns number of pairs

Method / Syntax	What it does
pair.first	The key of the pair
pair.second	The value of the pair

Program — Write a C++ program using map that stores four students name and marks.

```
#include<iostream>
#include<map>
using namespace std;

int main() {
    // Create a map: student name → marks
    map<string, int> marks;

    // Insert key-value pairs
    marks["Alice"] = 90;
    marks["Bob"]   = 85;
    marks["Carol"] = 92;
    marks["Dave"]  = 78;

    // Print all pairs (map auto-sorts by key)
    cout << "Student Marks:" << endl;
    for(auto m : marks) {
        cout << m.first << " -> " << m.second << endl;
    }

    // Access a specific student's marks
    cout << "Carol's marks: " << marks["Carol"] << endl;

    return 0;
}
```

▸ Output

```
Student Marks:
Alice -> 90
Bob -> 85
Carol -> 92
Dave -> 78
Carol's marks: 92
```

 Tip: Map automatically sorts by key alphabetically/numerically. Notice the output is in alphabetical order (Alice, Bob, Carol, Dave) even though we did not sort!

Algorithm — Quick Note

The `#include <algorithm>` header gives us ready-made functions that work on containers. These functions work with `begin()` and `end()` of the container. You don't need to write sorting or searching code yourself!

`Common functions: sort() binary_search() find() reverse() min() / max() count()`

Example: `sort(v.begin(), v.end())` sorts a vector in ascending order.

Iterator

An object that points to elements inside a container

Real-life Analogy

 Think of an iterator as the blinking cursor in a text editor. It always points to one character at a time, and you can move it forward (++) or backward (--) through the text.

Key Points

- An iterator is an object that works like a pointer
- Used to traverse (move through) container elements one by one
- v.begin() — points to the first element
- v.end() — points one past the last element (not the last element itself!)
- *it — dereference: gives the value at current position
- it++ — moves iterator to the next element

Iterator Syntax

Code	Meaning
`vector<int>::iterator it`	Declare an iterator for a vector of int
`it = v.begin()`	Point iterator to first element
`it != v.end()`	Loop condition until past last element
*it	Get the value at current position
it++	Move to next element
`auto it = v.begin()`	Shorthand — let compiler figure out the type

Program — Iterator#include<iostream>

#include<vector>

using namespace std;

```
int main() {
    // Create a vector
    vector<int> v = {10, 20, 30, 40, 50};

    // Declare an iterator
    vector<int>::iterator it;

    // Traverse using iterator
    cout << "Elements using iterator: ";
    for(it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";  // *it gives value at current position
    }
    cout << endl;

    // Print only first and last element
    it = v.begin();
    cout << "First element: " << *it << endl;

    it = v.end();
    it--;  // move one back from end to reach last element
    cout << "Last element: "  << *it << endl;

    return 0;
}
```

▶ **Output**

```
Elements using iterator: 10 20 30 40 50
First element: 10
Last element: 50
```

How the Iterator Moves Through the Vector

it position	*it value	Condition it != end()
`begin() → index 0`	10	true → print 10, it++
index 1	20	true → print 20, it++
index 2	30	true → print 30, it++

it position	*it value	Condition it != end()
index 3	40	true → print 40, it++
index 4	50	true → print 50, it++
`end() → past last`	—	false → loop stops

 Remember: v.end() does NOT point to the last element — it points one position after the last element. Always use it-- to go back to the last element from end().

Quick Summary Table

Container	Type	Header	Access	Key Feature
Vector	Sequence	`<vector>`	`v[i] by index`	Dynamic array — auto resize
List	Sequence	`<list>`	Iterator only	Doubly linked — fast insert/delete
Stack	Adaptor	`<stack>`	`top() only`	LIFO — last in, first out
Map	Associative	`<map>`	`m["key"] by key`	Key-value pairs, sorted by key
Iterator	—	any	`*it (dereference)`	Points to elements, moves with ++/--

Templates & Exception Handling

Templates	Exception Handling	Key Keywords	Code Examples

Part 1: Templates

Definition: A template is a blueprint for creating a family of classes or functions. It uses a placeholder type (T) that gets replaced by a specific data type at the time of use. Templates are also called parameterized classes or functions.

Types of Templates

C++ provides two kinds of templates:

- Class Template — used to create a generic class that works with multiple data types (e.g., an array class for int, float, char).
- Function Template — used to create a generic function that works with multiple types (e.g., a swap() function for any type T).

Class Template

Syntax

```
template<class T1, class T2>
class classname {
    T1 var1;
    T2 var2;
public:
    // constructor and methods
};
```

Key Term: Instantiation — the process of creating a specific class from a class template. E.g., Test<float, int> creates a version where T1 = float and T2 = int.

Example: Class Template with Two Type Parameters

```
#include <iostream>
using namespace std;

template<class T1, class T2>
class Test {
    T1-Float a=1.23;
    T2-int b=123;
public:
    Test(T1-Float x=1.23, T2-int y=123)
 {
a = x=1.23;
 b = y=123; }
    void show() {
        cout << a =1.23<< " and " << b=123 << "\n";
    }
};

int main() {
    Test<float, int>  test1(1.23, 123);   // Instantiation 1
    Test<int, char>   test2(100, 'W');    // Instantiation 2
    test1.show();  // Output: 1.23 and 123
    test2.show();  // Output: 100 and W
    return 0;
}
```

Function Template

Syntax

```
template<class T>
returntype functionname(T arg1, T arg2) {
```

```
    // body using type T
}
```

Example: Generic Swap Function

```
#include <iostream>
using namespace std;

template <class T>
void swap(T &x, T &y) {
    T temp = x;
    x = y;
    y = temp;
}

// Works for int:
swap(m, n);    // T is deduced as int

// Works for float:
swap(a, b);    // T is deduced as float
```

Programs:
Write a C++ program using Function Template:

```
#include <iostream>
using namespace std;

template <typename vit>
vit disp(vit a, vit b)
{
        return (a+b);
}

template <typename vit>
vit disp(vit a)
{
        cout<<a<<endl;
return a;
}

template <typename vit1, typename vit2>
vit2 disp(vit1 a, vit2 b)
{
        return (a+b);
}
int main()
```

```
{
        cout<<disp<int>(10,20)<<endl;
   cout<<disp<double, double>(10.2,20.2)<<endl;
   cout<<disp<int, double>(10,20.2)<<endl;
   disp<char>('A');
}
```

Write a C++ program using Class Template:

```
#include <iostream>
using namespace std;

template <typename vit>
class sample
{
        public:
        vit mark;
        sample(vit x)
      {
                mark = x;
        }
        void disp()
      {
                cout <<mark<<endl;
        }
};

int main()
{
        sample<int> s1(123);
   sample<double> s2(12.56);
   s1.disp();
   s2.disp();
        return 0;
}
```

Key Points Summary — Templates

Term	Meaning
template<class T>	Declares a template with type parameter T
Instantiation	Creating a specific class/function from a template
Parameterized	Another name for template-based classes or functions
Class template	Creates a family of classes (multiple data type versions)
Function template	Creates a family of functions (type-safe, compiler-deduced)

Exception Handling

Definition: Exceptions are run-time anomalies errors that occur during execution beyond normal logic or syntax errors. Examples include division by zero, array out-of-bounds access, and memory exhaustion.

Types of Exceptions

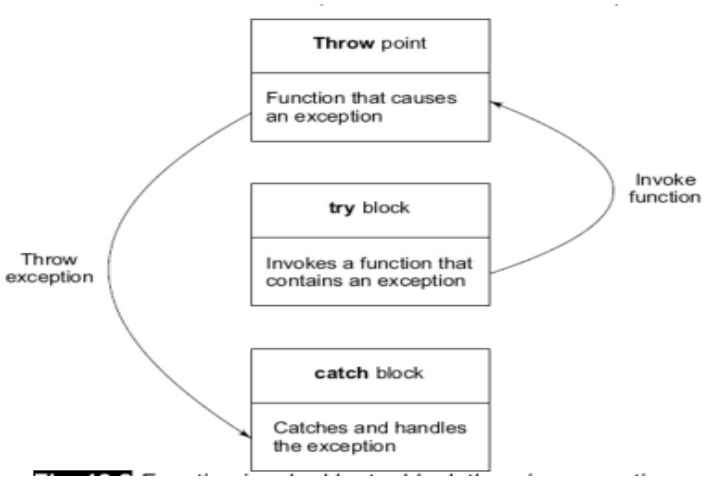
Synchronous Exceptions	Asynchronous Exceptions
Caused by the program itself Examples: divide by zero, out-of-range, overflow **Handled by C++**	Caused by external events Examples: keyboard interrupts, hardware signals **NOT handled by C++**

The Four Tasks of Exception Handling

1	**Hit**	Find the problem the exception occurs in the code
2	**Throw**	Inform that an error occurred using the throw statement
3	**Catch**	Receive the error information in the catch block
4	**Handle**	Take corrective action to recover from the error

The Three Keywords: try, throw, catch

- try : wraps a block of statements that may generate exceptions (the try block).
- throw :used inside the try block to throw an exception when detected. Exceptions are objects.
- catch : immediately follows the try block and handles the thrown exception.



Important Rule: The catch block must immediately follow the try block. If the thrown type matches the catch argument type, the catch block executes. If types do not match, the program aborts via abort(). If no exception is thrown, the catch block is skipped.

Exception Handling — Syntax

```
try {
    // Block of statements that may throw an exception
    throw exception;    // throws an object
}
catch(type arg) {
    // Block that handles the exception
}
```

Example: Divide by Zero

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter values of a and b: ";
    cin >> a >> b;

    int x = a - b;
    try {
        if (x != 0)
            cout << "Result (a/x) = " << a/x << "\n";
        else
            throw(x);         // throws int object
    }
    catch(int i) {            // catches int exception
        cout << "Exception caught: DIVIDE BY ZERO\n";
    }
    cout << "END";
    return 0;
}

// Run 1: a=20, b=15  → Result (a/x) = 4
// Run 2: a=10, b=10  → Exception caught: DIVIDE BY ZERO
```

Key Points Summary — Exception Handling

Term	Meaning
try	Prefixes a block that may generate exceptions
throw	Throws an exception (object) when an error is detected

catch(type)	Catches and handles the thrown exception by type match
abort()	Called automatically when thrown/catch types don't match
Synchronous	Errors caused by the program — handled by C++
Asynchronous	Errors from external events — NOT handled by C++

Quick Reference Card

Template Cheat Sheet

```
// Class Template
template<class T>
class MyClass { T data; };
MyClass<int> obj;          // instantiation

// Function Template
template<class T>
void myFunc(T x) { /* ... */ }
myFunc(42);               // T deduced as int
myFunc(3.14);              // T deduced as double
```

Exception Handling Cheat Sheet

```
try {
    // risky code
    if (error) throw errorObject;
}
catch(ErrorType e) {
    // handle error
}
// execution continues here (catch block is skipped if no error)
```

> **Remember:** Templates = compile-time feature (no runtime cost). Exception handling = runtime feature (detects errors during execution). Both were added to ANSI C++ to improve code safety and reusability.